macOS Example System and Kernel Log Queries

youtube.com/13cubed

Retrieve log entries whose process is set to "kernel," restricted to the past hour:

```
log show --predicate 'process == "kernel"' --last 1h
```

Refine search to filter kext-related events in the kernel logs:

```
log show --predicate 'process == "kernel" AND eventMessage CONTAINS "kext"' --last 1h
```

Stream kernel messages in real time (resource-intensive):

```
log stream --predicate 'process == "kernel"' --info
```

Pull recent messages from the launchd subsystem (within the last 30 minutes):

```
log show --predicate 'subsystem == "com.apple.launchd"' --last 30m --info --style syslog
```

Search for kernel events referencing sandbox or AMFI checks:

```
log show --predicate '(process == "kernel") AND (eventMessage CONTAINS "sandbox" OR eventMessage CONTAINS "AMFI")' --last 1h
```

Redirect kernel logs to a file for later analysis:

```
log show --predicate 'process == "kernel"' --last 1h > kernel_events_last_hour.txt
```

macOS Example Authentication and Security Log Queries

youtube.com/13cubed

Retrieve log entries for the "loginwindow" subsystem (last hour):
```
log show --predicate 'subsystem == "com.apple.loginwindow"' --last 1h
```
Also query `com.apple.loginwindow.logging` for additional log entries

Refine the search to show only successful or failed login attempts (run each command separately as needed):
```
log show --predicate 'subsystem == "com.apple.loginwindow" AND eventMessage CONTAINS "success"' --last 1h
log show --predicate 'subsystem == "com.apple.loginwindow" AND eventMessage CONTAINS "failed"' --last 1h
```

An alternative approach is to look for "fail", which would match "failed" and "failure":
```
log show --predicate 'subsystem == "com.apple.loginwindow" AND eventMessage CONTAINS "fail"' --last 1h
```

Locate entries related to sudo or PAM (privilege escalations):
```
log show --predicate '(process == "sudo") OR (eventMessage CONTAINS "PAM")' --last 1h
```

Stream authentication events in real time (resource-intensive, live system only):
```
log stream --predicate 'subsystem == "com.apple.loginwindow" OR process == "sudo"' --info
```

Pull Gatekeeper logs (syspolicyd) from the last 24 hours:
```
log show --predicate 'process == "syspolicyd"' --last 24h
```

Pull Gatekeeper logs (spctl) from the last 24 hours (older versions of macOS):
```
log show --predicate 'process == "spctl"' --last 24h
```

Inspect the security assessment subsystem (e.g., Gatekeeper decisions):
```
log show --predicate 'subsystem == "com.apple.security.assessment"' --last 24h
```

Search for TCC "deny" events in the last hour:
```
log show --predicate 'eventMessage CONTAINS "TCC" AND eventMessage CONTAINS "deny"' --last 1h
```

Redirect authentication-related logs to a file for correlation (last hour):
```
log show --predicate 'subsystem == "com.apple.loginwindow" OR process == "sudo"' --last 1h > auth_events_last_hour.txt
```

macOS Example Advanced Authentication and Security Log Queries

youtube.com/13cubed

Retrieve log entries for authd from the last hour:

```
log show --predicate 'process == "authd"' --last 1h
```

Specify a custom time window (example using January 1, 2025, from 10:00 to 11:00):

```
log show --predicate 'process == "authd"' --start '2025-01-01 10:00:00' --end '2025-01-01 11:00:00'
```

Show log entries for opendirectoryd from the last hour:

```
log show --predicate 'subsystem == "com.apple.opendirectoryd"' --last 1h
```

Show log entries for sshd from the last hour:

```
log show --predicate 'process == "sshd"' --last 1h
```

Search for log entries containing "ssh" from the last hour:

```
log show --predicate 'eventMessage CONTAINS[c] "ssh"' --last 1h --info
```

Show accepted/failed SSH logins from the last hour:

```
log show --predicate 'processImagePath CONTAINS[c] "sshd" && (eventMessage CONTAINS[c] "accepted" || eventMessage CONTAINS[c]
"error")' --last 1h --info
```

Pull log entries for securityd from the last hour:

```
log show --predicate 'process == "securityd"' --last 1h
```

macOS Example Firewall and Proxy Log Queries

Check the status of the macOS firewall:

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate
```

View ALF events from the last 24 hours:

```
log show --predicate '(process == "socketfilterfw") OR (subsystem == "com.apple.alf")' --last 24h
```

Monitor events in real time on a live system (resource-intensive):

```
log stream --predicate '(process == "socketfilterfw") OR (subsystem == "com.apple.alf")'
```

Preserve firewall logs by saving them to a file:

```
log show --predicate '(process == "socketfilterfw") OR (subsystem == "com.apple.alf")' --last 24h > firewall_snapshot.txt
```

Look for references to proxy configurations (last 24 hours):

```
log show --predicate 'eventMessage CONTAINS "proxy"' --last 24h
```

Narrow the proxy search to a particular daemon or subsystem:

```
log show --predicate 'process == "configd" AND eventMessage CONTAINS "proxy"' --last 24h
```

macOS Example Wi-Fi and Network Log Queries

View Wi-Fi logs for airportd (last 24 hours):
```
log show --predicate 'process == "airportd"' --last 24h
```

Refine Wi-Fi logs to look for activity associated with a specific SSID:
```
log show --predicate 'process == "airportd" AND eventMessage CONTAINS "SSID-NAME-HERE"' --last 24h
```

Monitor Wi-Fi events in real time (resource-intensive):
```
log stream --predicate 'process == "airportd"' --info
```

Look for network configuration changes via configd:
```
log show --predicate 'process == "configd"' --last 24h
```

Check broader network events on newer macOS releases:
```
log show --predicate 'subsystem == "com.apple.network"' --last 24h
```

Look for local service discovery with mDNSResponder:
```
log show --predicate 'process == "mDNSResponder"' --last 24h
```

Search rapportd logs for device continuity events:
```
log show --predicate 'process == "rapportd"' --last 24h
```

For detailed logs covering AirDrop activity:
```
log show --predicate 'subsystem == "com.apple.sharing"' --last 24h
log show --predicate 'process == "sharingd"' --last 24h
```

Look for VPN or advanced network features (NetworkExtension subsystem):
```
log show --predicate 'subsystem == "com.apple.networkextension"' --last 24h
```

Look for VPN or advanced network features (nehelper process):
```
log show --predicate 'process == "nehelper"' --last 24h
```

macOS Example Wi-Fi and Network Log Queries (Continued)

Preserve a snapshot of key network events:

```
log show --predicate '(process == "airportd" OR process == "configd" OR process == "rapportd" OR subsystem ==
"com.apple.network")' --last 24h > network_events_snapshot.txt
```

macOS Example Bluetooth Log Queries

youtube.com/13cubed

Show Bluetooth logs from bluetoothd or blued (last 24 hours):

```
log show --predicate 'process == "bluetoothd" OR process == "blued"' --last 24h
```

Filter for pairing-related messages:

```
log show --predicate '(process == "bluetoothd" OR process == "blued") AND eventMessage CONTAINS "Pairing"' --last 24h
```

Filter for connected/disconnected messages:

```
log show --predicate '(process == "bluetoothd" OR process == "blued") AND (eventMessage CONTAINS "Connected" OR eventMessage
CONTAINS "Disconnected")' --last 24h
```

Check wirelessproxd logs (Apple Watch unlock, proximity features):

```
log show --predicate 'process == "wirelessproxd"' --last 24h
```

Search rapportd logs for continuity over Bluetooth:

```
log show --predicate 'process == "rapportd"' --last 24h
```

Look at subsystem-level Bluetooth events:

```
log show --predicate 'subsystem == "com.apple.Bluetooth"' --last 24h
```

Stream Bluetooth logs in real time (resource-intensive):

```
log stream --predicate '(process == "bluetoothd" OR process == "blued" OR process == "wirelessproxd")' --info
```

Preserve a snapshot of Bluetooth events:

```
log show --predicate '(process == "bluetoothd" OR process == "blued" OR process == "wirelessproxd" OR subsystem ==
"com.apple.Bluetooth")' --last 24h > bluetooth_events_snapshot.txt
```

macOS Example Gatekeeper, TCC, and XProtect Log Queries

youtube.com/13cubed

Gatekeeper (syspolicyd) logs (last 24 hours):
```
log show --predicate 'process == "syspolicyd"' --last 24h
```

Older references (spctl) for Gatekeeper (rare on modern macOS):
```
log show --predicate 'process == "spctl"' --last 24h
```

Basic TCC events (last 24 hours):
```
log show --predicate 'eventMessage CONTAINS "TCC"' --last 24h
```

Filter for TCC denials:
```
log show --predicate 'eventMessage CONTAINS "TCC" AND eventMessage CONTAINS "deny"' --last 24h
```

Check for TCC "grant" messages:
```
log show --predicate 'eventMessage CONTAINS "TCC" AND eventMessage CONTAINS "grant"' --last 24h
```

Search for XProtect references (last 24 hours):
```
log show --predicate 'eventMessage CONTAINS "XProtect"' --last 24h
```

Check the XprotectFramework subsystem (legacy references):
```
log show --predicate 'subsystem == "com.apple.XprotectFramework"' --last 24h
```

Look for new XProtect Remediator logs:
```
log show --predicate 'eventMessage CONTAINS "xprotectservice" OR eventMessage CONTAINS "XProtectService"' --last 24h
```

Monitor Gatekeeper, TCC, and XProtect in real time (resource-intensive):
```
log stream --predicate '(process == "syspolicyd") OR (eventMessage CONTAINS "TCC") OR (eventMessage CONTAINS "XProtect") OR
(eventMessage CONTAINS "xprotectservice")' --info
```

Preserve Gatekeeper, TCC, and XProtect logs (last 24 hours):
```
log show --predicate '(process == "syspolicyd") OR (eventMessage CONTAINS "TCC") OR (eventMessage CONTAINS "XProtect") OR
(eventMessage CONTAINS "xprotectservice")' --last 24h > gatekeeper_tcc_xprotect_snapshot.txt
```

macOS Example Crash Reporting Log Queries

youtube.com/13cubed

Show CrashReporter subsystem logs (last 24 hours):

```
log show --predicate 'subsystem == "com.apple.crashreporter"' --last 24h
```

Search for "crash" or "exception" keywords:

```
log show --predicate 'eventMessage CONTAINS "crash" OR eventMessage CONTAINS "exception"' --last 24h
```

Monitor crash-related events in real time (resource-intensive):

```
log stream --predicate 'subsystem == "com.apple.crashreporter" OR eventMessage CONTAINS "crash"' --info
```

Check spindump references (hung processes/spins):

```
log show --predicate 'process == "spindump"' --last 24h
```

Look for diagnosticd logs (coordinates crash/diagnostic data):

```
log show --predicate 'process == "diagnosticd"' --last 24h
```

Search for kernel panics:

```
log show --predicate 'eventMessage CONTAINS "panic" OR subsystem == "com.apple.kext"' --last 24h
```

Preserve crash-related logs:

```
log show --predicate 'subsystem == "com.apple.crashreporter" OR process == "spindump" OR process == "diagnosticd"' --last 24h >
crash_logs_snapshot.txt
```

macOS Legacy Logs and Locations

youtube.com/13cubed

system.log (Legacy main system log):
`/var/log/system.log`

Compressed archives of system.log (historical data):
`/var/log/system.log.*` (e.g., system.log.0.gz)

ASL or DiagnosticMessages (older ASL data):
`/var/log/asl/` or `/var/log/DiagnosticMessages/`

Other log files:
`/var/log/wifi.log`
`/var/log/racoon.log`
`/var/log/ppp.log`

ASL configuration file:
`/etc/asl.conf`

View system.log:
```
less /var/log/system.log
```

Search system.log:
```
grep "keyword" /var/log/system.log
```

Check rotation archives:
```
ls -lh /var/log/system.log.*
```

Examine wifi.log, ppp.log, or racoon.log similarly:
```
less /var/log/wifi.log
```

Parse .asl files on older macOS versions:
```
syslog -r -f /var/log/asl/filename.asl
```

macOS Legacy Logs and Locations (Continued)

Search system.log for suspicious terms:

```
grep -i "error" /var/log/system.log
```

Check archived logs for historical data:

```
ls -lh /var/log/system.log.*
```

Browse active logging:

```
tail -f /var/log/system.log
```

Look for ASL databases:

```
ls -lh /var/log/asl/
```

Preserve logs before rotation:

```
cp /var/log/system.log /safe_location/
cp /var/log/system.log.0.gz /safe_location/
```

macOS Example Application-specific Log Queries and Locations

Little Snitch – View Logs:
`littlesnitch log`

Little Snitch – View Real-Time Traffic Logs:
`littlesnitch log-traffic`

LuLu – Stream Logs from the Unified Logs:
`log stream --predicate="subsystem=='com.objective-see.lulu'" --debug`

LuLu – Older Versions or Artifacts:
`/Library/Objective-See/LuLu`
`/Library/Logs/LuLu`

Jamf Connect – Show Historical Logs (Last 30 Min):
`log show --predicate 'subsystem == "com.jamf.connect"' --debug --last 30m > ~/Desktop/JamfConnect.log`

Jamf Connect – Stream Logs in Real Time:
`log stream --predicate 'subsystem == "com.jamf.connect"' --debug`

Jamf Pro – Plain Text Logs:
`/Library/Logs/Jamf/`
`/usr/local/jamf/logs/`
`/var/log/jamf.log` (primary log used by Jamf Pro)

Antivirus / Endpoint Security:
`/Library/Logs/<VendorName>/`

VPN Clients (Non-Apple):
`/Library/Logs/<Vendor>/`
`~/Library/Logs/<Vendor>/`

macOS Example Application-specific Log Queries and Locations (Continued)

Browser / Chat Apps:
`~/Library/Application Support/<AppName>/`

System-Wide Logs Location:
`/Library/Logs`

User-Specific Logs Location:
`~/Library/Logs`

Logical Operators and Predicate Basics for Logs

youtube.com/13cubed

Logical Operators:
AND
OR
NOT

Example:
```
log show --predicate '(process == "mds") OR (process == "mdworker")'
```

Comparison Operators:
==
!=
<
>
<=
>=

Example:
```
log show --predicate 'process == "kernel"' --last 1h
```

String Matching:
CONTAINS
CONTAINS[c]
BEGINSWITH
ENDSWITH
LIKE

Example:
```
log show --predicate 'eventMessage CONTAINS[c] "indexing"'
```

Other Common Options for Logs

`--color`
Adds colorized output in Terminal (`auto`, `always`, `none`)

`--debug`
Includes debug-level messages (more verbose)

`--end`
Specifies an absolute end time window (e.g., `--end '2025-02-01 10:00:00'`)

`--info`
Includes info messages (often hidden by default)

`--last`
Specifies a relative time window (e.g., `--last 24h`, `--last 30m`)

`--level`
Specify which log level to show (`default`, `info`, `debug`, etc.)

`--predicate`
Filters log entries by an NSPredicate-like statement

`--source`
Displays source info for the entries (verbose)

`--start`
Specifies an absolute start time window (e.g., `--start '2025-02-01 08:00:00'`)

`--style`
Specify the message style (`syslog`, `json`, `compact`)